MCSC-202 (Simplified) - utils.py Python Source File

```python
# Import the math module to use mathematical functions like sin, cos, exp, log
import math

def evaluate(func_str, x_val):
    # Create a dictionary of allowed mathematical functions from the math module
    # k: v creates key-value pairs (e.g. 'sin': math.sin) for non-private math members
    math_env = {k: v for k, v in math.__dict__.items() if not k.startswith("__")}

    # Add 'x' to the environment, pointing it to the current value of x_val
    math_env["x"] = x_val

    # Safely evaluate the mathematical expression string func_str
    # {"__builtins__": {}} blocks access to standard built-in functions (like open, import)
    # math_env serves as the namespace containing local variables (x) and math library functions
    return eval(func_str, {"__builtins__": {}}, math_env)

# Executable test block for utils
if __name__ == "__main__":
    print("=== Testing Utility Evaluator ===")
    val = evaluate("x**2 + sin(x)", 0)
    print(f"evaluate('x**2 + sin(x)', 0) = {val} (expected: 0.0)")

    val2 = evaluate("cos(x) + exp(x)", 0)
    print(f"evaluate('cos(x) + exp(x)', 0) = {val2} (expected: 2.0)")
    print("================================")
```

MCSC-202 (Simplified) - absolute_relative_error.py Python Source File

```python
def calculate_errors(true_val, approx_val):
    # Absolute error: magnitude of difference between true and approximate value
    absolute_error = abs(true_val - approx_val)

    # Relative error: absolute error divided by the magnitude of the true value
    # We check if true_val is zero to avoid division by zero
    relative_error = absolute_error / abs(true_val) if true_val != 0 else 0.0

    # Percentage error: relative error multiplied by 100 to convert to percentage
    percentage_error = relative_error * 100.0

    return absolute_error, relative_error, percentage_error

# Executable test block
if __name__ == "__main__":
    print("=== Testing Absolute, Relative & Percentage Errors ===")
    true_val = 10.0
    approx_val = 9.8
    ea, er, ep = calculate_errors(true_val, approx_val)
    print(f"True value: {true_val}, Approximate value: {approx_val}")
    print(f"Absolute error: {ea} (expected: 0.2)")
    print(f"Relative error: {er} (expected: 0.02)")
    print(f"Percentage error: {ep}% (expected: 2.0%)")
    print("=====================================================")
```

```python
import math

def truncation_error_bound(max_deriv, n, x, a):
    # Formula for maximum truncation error bound of nth degree Taylor polynomial:
    # Error <= (M / n!) * |x - a|^n
    # where M is the maximum value of the nth derivative on the interval [a, x]

    # Compute n factorial (n!)
    fact = math.factorial(n)

    # Compute |x - a|^n
    diff_pow = abs(x - a) ** n

    # Calculate the upper bound of the truncation error
    error_bound = (max_deriv / fact) * diff_pow

    return error_bound

# Executable test block
if __name__ == "__main__":
    print("=== Testing Truncation Error Bound ===")
    max_deriv = 2.0
    n = 3
    x = 1.1
    x0 = 1.0
    err_bound = truncation_error_bound(max_deriv, n, x, x0)
    print(f"Truncation error bound (n={n}, dx={x-x0:.2f}, max_deriv={max_deriv}):")
    print(f"Bound = {err_bound} (expected: ~0.000333)")
    print("=====================================")
```

MCSC-202 (Simplified) - bisection.py Python Source File

```python
def bisection(f, a, b, tol=1e-6, max_iter=100):
    # Check if a root exists in the given interval [a, b]
    # Intermediate Value Theorem requires f(a) and f(b) to have opposite signs
    if f(a) * f(b) >= 0:
        raise ValueError("Root not guaranteed. f(a) and f(b) must have opposite signs.")

    # Iterate up to the maximum number of iterations
    for i in range(max_iter):
        # Find the midpoint (c) of the current interval [a, b]
        c = (a + b) / 2.0

        # Check if we have met the tolerance limit or if c is an exact root
        if abs(b - a) / 2.0 < tol or f(c) == 0:
            return c # Return the computed root

        # Decide which subinterval contains the root
        if f(a) * f(c) < 0:
            b = c # The root lies in the left subinterval [a, c], so update upper bound
        else:
            a = c # The root lies in the right subinterval [c, b], so update lower bound

    return (a + b) / 2.0 # Return the best estimate if max_iter is reached

# Executable test block
if __name__ == "__main__":
    print("=== Testing Bisection Method ===")
    f = lambda x: x**2 - 2
    root = bisection(f, 1, 2)
    print(f"Bisection root of x**2 - 2 in [1, 2]: {root:.6f} (expected: ~1.414214)")
    print("================================")
```

```python
def false_position(f, a, b, tol=1e-6, max_iter=100):
    # Check if a root exists in the given interval [a, b]
    # Intermediate Value Theorem requires f(a) and f(b) to have opposite signs
    if f(a) * f(b) >= 0:
        raise ValueError("Root not guaranteed. f(a) and f(b) must have opposite signs.")

    fa = f(a)
    fb = f(b)
    c = a # Initialize c

    # Iterate up to the maximum number of iterations
    for i in range(max_iter):
        # Calculate the false position midpoint c using the formula:
        # c = (a * f(b) - b * f(a)) / (f(b) - f(a))
        c = (a * fb - b * fa) / (fb - fa)
        fc = f(c)

        # Check if we have met the tolerance limit or if c is an exact root
        if abs(fc) < tol or fc == 0:
            return c # Return the computed root

        # Decide which subinterval contains the root and update the bounds
        if fa * fc < 0:
            b = c # Root lies in [a, c]
            fb = fc
        else:
            a = c # Root lies in [c, b]
            fa = fc

    return c # Return the best estimate if max_iter is reached

# Executable test block
if __name__ == "__main__":
    print("=== Testing False Position Method ===")
    f = lambda x: x**2 - 2
    root = false_position(f, 1.0, 2.0)
    print(f"False Position root of x**2 - 2 in [1, 2]: {root:.6f} (expected: ~1.414214)")
    print("====================================")
```

```python
def newton_raphson(f, df, x0, tol=1e-6, max_iter=100):
    x = x0 # Set initial guess

    # Iterate up to the maximum number of iterations
    for i in range(max_iter):
        fx = f(x) # Evaluate the function at the current point x
        dfx = df(x) # Evaluate the derivative at the current point x

        # Avoid division by zero (tangent is parallel to x-axis)
        if dfx == 0:
            raise ValueError("Derivative is zero. Newton-Raphson method fails.")

        # Calculate the next estimate using Newton's formula: x_new = x - f(x)/f'(x)
        x_new = x - fx / dfx

        # Check if the difference between successive estimates is within tolerance
        if abs(x_new - x) < tol:
            return x_new # Return the converged root

        # Update the current point for the next iteration
        x = x_new

    return x # Return the best estimate if max_iter is reached

# Executable test block
if __name__ == "__main__":
    print("=== Testing Newton-Raphson Method ===")
    f = lambda x: x**2 - 2
    df = lambda x: 2*x
    root = newton_raphson(f, df, 1.5)
    print(f"Newton-Raphson root of x**2 - 2 with x0=1.5: {root:.6f} (expected: ~1.414214)")
    print("=====================================")
```

MCSC-202 (Simplified) - secant.py Python Source File

```python
def secant(f, x0, x1, tol=1e-6, max_iter=100):
    # Iterate up to the maximum number of iterations
    for i in range(max_iter):
        fx0 = f(x0) # Evaluate function at x0
        fx1 = f(x1) # Evaluate function at x1

        # Avoid division by zero when function values are identical
        if fx1 - fx0 == 0:
            raise ValueError("Division by zero. Secant method fails.")

        # Calculate the next estimate using the Secant formula
        x_next = x1 - fx1 * (x1 - x0) / (fx1 - fx0)

        # Check if the difference between successive estimates is within tolerance
        if abs(x_next - x1) < tol:
            return x_next # Return the converged root

        # Update the previous two points for the next iteration
        x0 = x1
        x1 = x_next

    return x1 # Return the best estimate if max_iter is reached

# Executable test block
if __name__ == "__main__":
    print("=== Testing Secant Method ===")
    f = lambda x: x**2 - 2
    root = secant(f, 1.0, 2.0)
    print(f"Secant root of x**2 - 2 with guesses 1.0 & 2.0: {root:.6f} (expected: ~1.414214)")
    print("==============================")
```

```python
def lu_decomposition(A, b):
    n = len(A) # Get the size of the matrix A (number of rows/columns)

    # Initialize L as an identity matrix (1s on diagonal, 0s elsewhere)
    L = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
    # Initialize U as a zero matrix of size n x n
    U = [[0.0 for _ in range(n)] for _ in range(n)]

    # LU Decomposition (Doolittle's Algorithm)
    for i in range(n):
        # Calculate elements of the Upper triangular matrix U
        for k in range(i, n):
            # U[i][k] = A[i][k] - sum_{j=0}^{i-1} (L[i][j] * U[j][k])
            sum_val = sum(L[i][j] * U[j][k] for j in range(i))
            U[i][k] = A[i][k] - sum_val

        # Calculate elements of the Lower triangular matrix L
        for k in range(i + 1, n):
            # L[k][i] = (A[k][i] - sum_{j=0}^{i-1} (L[k][j] * U[j][i])) / U[i][i]
            sum_val = sum(L[k][j] * U[j][i] for j in range(i))
            L[k][i] = (A[k][i] - sum_val) / U[i][i]

    # Forward Substitution to solve Ly = b
    y = [0.0] * n # Initialize vector y with zeros
    for i in range(n):
        # y[i] = b[i] - sum_{j=0}^{i-1} (L[i][j] * y[j])
        sum_val = sum(L[i][j] * y[j] for j in range(i))
        y[i] = b[i] - sum_val

    # Back Substitution to solve Ux = y
    x = [0.0] * n # Initialize solution vector x with zeros
    for i in range(n - 1, -1, -1): # Iterate backwards from n-1 down to 0
        # x[i] = (y[i] - sum_{j=i+1}^{n-1} (U[i][j] * x[j])) / U[i][i]
        sum_val = sum(U[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (y[i] - sum_val) / U[i][i]

    return L, U, x # Return L, U matrices and the solution vector x

# Executable test block
if __name__ == "__main__":
    print("=== Testing LU Decomposition ===")
    A = [[2.0, -1.0, 0.0],
         [-1.0, 2.0, -1.0],
         [0.0, -1.0, 2.0]]
    b = [1.0, 0.0, 1.0]
    L, U, x = lu_decomposition(A, b)
    print("Matrix A:")
    for row in A:
        print(row)
    print("Vector b:", b)
    print("L matrix:")
    for row in L:
        print([round(val, 4) for val in row])
    print("U matrix:")
    for row in U:
        print([round(val, 4) for val in row])
    print(f"Solution x: {x} (expected: [1.0, 1.0, 1.0])")
    print("================================")
```

```python
def thomas_algorithm(a, b, c, d):
    n = len(d) # Get the size of the system (number of equations)

    cp = [0.0] * n # Initialize array to store modified upper diagonal coefficients c'
    dp = [0.0] * n # Initialize array to store modified RHS values d'

    # Initialize the first values of c' and d'
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]

    # Forward elimination pass
    for i in range(1, n):
        # Calculate denominator for c' and d' updates
        denom = b[i] - a[i] * cp[i - 1]

        # Update upper diagonal coefficient c' (for all but the last equation)
        if i < n - 1:
            cp[i] = c[i] / denom

        # Update RHS value d'
        dp[i] = (d[i] - a[i] * dp[i - 1]) / denom

    # Back substitution to compute the solution vector x
    x = [0.0] * n # Initialize solution vector x with zeros
    x[-1] = dp[-1] # The last element of x is simply the last element of d'
    for i in range(n - 2, -1, -1): # Iterate backwards from n-2 down to 0
        # Compute the value of x[i] using the relation: x[i] = d'[i] - c'[i] * x[i+1]
        x[i] = dp[i] - cp[i] * x[i + 1]

    return x # Return the final solution vector x

# Executable test block
if __name__ == "__main__":
    print("=== Testing Thomas Algorithm ===")
    # System:
    # 2*x0 - x1 = 1
    # -x0 + 2*x1 - x2 = 0
    # -x1 + 2*x2 = 1
    a = [0.0, -1.0, -1.0]
    b_diag = [2.0, 2.0, 2.0]
    c = [-1.0, -1.0, 0.0]
    d = [1.0, 0.0, 1.0]
    x = thomas_algorithm(a, b_diag, c, d)
    print(f"Thomas Algorithm Solution: {x} (expected: [1.0, 1.0, 1.0])")
    print("================================")
```

```python
def forward_difference_table(x, y):
    n = len(y) # Get the number of data points

    # Initialize a 2D table of size n x n with zeros
    table = [[0.0 for _ in range(n)] for _ in range(n)]

    # Fill the first column of the table with y values (0th difference column)
    for i in range(n):
        table[i][0] = y[i]

    # Build the difference table column by column (1st difference up to (n-1)th difference)
    for j in range(1, n):
        for i in range(n - j):
            # Difference is the value below-left minus value current-left
            table[i][j] = table[i + 1][j - 1] - table[i][j - 1]

    return table # Return the complete difference table


def newton_forward_interpolation(x, y, target):
    n = len(x) # Get number of data points
    table = forward_difference_table(x, y) # Generate the forward difference table

    h = x[1] - x[0] # Calculate equal spacing interval h between x values
    u = (target - x[0]) / h # Calculate the normalized variable u for target interpolation point

    result = table[0][0] # Start with the first term of the formula (y0)
    u_term = 1.0 # Initialize variable to keep track of u product terms: u * (u-1) * (u-2)...
    fact = 1 # Initialize variable for factorial values: i!

    # Calculate terms for Newton's forward interpolation formula
    for i in range(1, n):
        u_term = u_term * (u - (i - 1)) # Compute u * (u - 1) * ... * (u - i + 1)
        fact = fact * i # Compute factorial of i: i!
        # Add term: (u_term / i!) * Delta^i y0
        result = result + (u_term / fact) * table[0][i]

    return result # Return the final interpolated value

# Executable test block for finite differences
if __name__ == "__main__":
    print("=== Testing Finite Difference & Interpolation ===")

    # Data points for y = 2**x + x**2
    x_pts = [0.0, 1.0, 2.0]
    y_pts = [1.0, 3.0, 9.0]

    # 1. Forward Difference Table
    print("\nData points:")
    for xi, yi in zip(x_pts, y_pts):
        print(f"x = {xi}, y = {yi}")

    table = forward_difference_table(x_pts, y_pts)
    print("\nForward Difference Table:")
    for idx, row in enumerate(table):
        # Print only the valid difference columns for each row
        print(f"Row {idx}: {[round(val, 4) for val in row[:len(table)-idx]]}")

    # 2. Newton's Forward Interpolation
    target = 1.5
    y_interp = newton_forward_interpolation(x_pts, y_pts, target)
    print(f"\nNewton Forward Interpolated Value at x = {target}: {y_interp} (expected: 5.5)")
    print("================================================")
```

```python
def backward_difference_table(y_vals):
    n = len(y_vals)
    # Initialize a 2D list with zeros of size n x n
    table = [[0.0 for _ in range(n)] for _ in range(n)]

    # Fill the first column with y values
    for i in range(n):
        table[i][0] = y_vals[i]

    # Calculate backward differences column by column
    for j in range(1, n):
        # Row index ranges from n-1 down to j
        for i in range(n - 1, j - 1, -1):
            table[i][j] = table[i][j - 1] - table[i - 1][j - 1]

    return table

def newton_backward_interpolation(x_vals, y_vals, x):
    n = len(x_vals)
    h = x_vals[1] - x_vals[0] # Calculate step size h (assuming uniform spacing)

    # Generate the backward difference table
    table = backward_difference_table(y_vals)

    # Calculate interpolation parameter p relative to the last point x_n (index n-1)
    p = (x - x_vals[n - 1]) / h

    # Start with the last y value: y_n
    interpolated_value = table[n - 1][0]
    p_term = 1.0
    factorial = 1.0

    # Sum the terms: y_n + p * del(y_n) + p*(p+1)/2! * del^2(y_n) + ...
    for j in range(1, n):
        p_term *= (p + j - 1)  # Compute p * (p + 1) * ... * (p + j - 1)
        factorial *= j         # Compute j!
        # Add the j-th term to the result
        interpolated_value += (p_term / factorial) * table[n - 1][j]

    return table, interpolated_value

# Executable test block
if __name__ == "__main__":
    print("=== Testing Newton Backward Interpolation ===")
    x_vals = [1.0, 1.2, 1.4, 1.6]
    y_vals = [2.7183, 3.3201, 4.0552, 4.9530]

    # Interpolate near the end of the table, e.g., x = 1.5
    target = 1.5
    table, result = newton_backward_interpolation(x_vals, y_vals, target)

    print("Backward Difference Table:")
    for i in range(len(y_vals)):
        row_str = " ".join(f"{table[i][j]:.4f}" for j in range(i + 1))
        print(f"Row {i} (x={x_vals[i]}): {row_str}")

    print(f"\nInterpolated value at x = {target}: {result:.4f} (expected: ~4.4817)")
    print("=============================================")
```

```python
import math

def forward_difference_table(y_vals):
    n = len(y_vals)
    # Initialize table with zeros
    table = [[0.0 for _ in range(n)] for _ in range(n)]

    # Fill first column with y values
    for i in range(n):
        table[i][0] = y_vals[i]

    # Compute forward differences column by column
    for j in range(1, n):
        for i in range(n - j):
            table[i][j] = table[i + 1][j - 1] - table[i][j - 1]

    return table

def stirling_interpolation(x_vals, y_vals, x):
    n = len(x_vals)
    h = x_vals[1] - x_vals[0] # Assuming equally spaced intervals

    # Generate the forward difference table
    table = forward_difference_table(y_vals)

    # Find the origin index (closest x_0 to target x)
    origin_idx = min(range(n), key=lambda i: abs(x_vals[i] - x))
    x0 = x_vals[origin_idx]

    # Calculate interpolation parameter p
    p = (x - x0) / h

    # Initialize Stirling estimate with y_0 (at the origin index)
    interpolated_value = table[origin_idx][0]

    # Loop over difference columns (orders)
    for k in range(1, n):
        if k % 2 == 1:
            # Odd order k = 2m + 1
            m = k // 2
            # Check table boundary constraints for required rows
            row1 = origin_idx - m
            row2 = origin_idx - m - 1
            if row1 < 0 or row2 < 0 or row1 >= n - k or row2 >= n - k:
                break # Stop if indices go out of bounds

            # Stirling coefficient for odd order: p * (p^2 - 1^2) * ... * (p^2 - m^2) / k!
            coeff = p
            for i in range(1, m + 1):
                coeff *= (p**2 - i**2)
            coeff /= math.factorial(k)

            # Average the two adjacent differences
            avg_diff = (table[row1][k] + table[row2][k]) / 2.0
            interpolated_value += coeff * avg_diff

        else:
            # Even order k = 2m
            m = k // 2
            row = origin_idx - m
            if row < 0 or row >= n - k:
                break # Stop if index goes out of bounds

            # Stirling coefficient for even order: p^2 * (p^2 - 1^2) * ... * (p^2 - (m-1)^2) / k!
            coeff = p**2
            for i in range(1, m):
                coeff *= (p**2 - i**2)
```

```
            coeff /= math.factorial(k)

            # Add the even difference term
            interpolated_value += coeff * table[row][k]

    return interpolated_value

# Executable test block
if __name__ == "__main__":
    print("=== Testing Stirling Central Interpolation ===")
    # Table of values for cos(x)
    x_vals = [0.10, 0.15, 0.20, 0.25, 0.30]
    y_vals = [0.9950, 0.9888, 0.9801, 0.9689, 0.9553]

    target = 0.17
    result = stirling_interpolation(x_vals, y_vals, target)
    print(f"Stirling estimation of cos({target}): {result:.6f} (expected: ~0.9856)")
    print("================================================")
```